

Note: Problema delle 8 Regine in C#

Con focus su LINQ, Lambda Functions e Operazioni su Dati

Panoramica del Programma

Il programma risolve il **Problema delle 8 Regine** usando **backtracking ricorsivo**.
Trovate 12 soluzioni uniche (eliminando i duplicati per simmetria).

Concetti Fondamentali: Backtracking Ricorsivo

Cos'è il Backtracking?

Il backtracking è una tecnica che:

1. **Scende** nel problema (aumenta la profondità)
2. **Torna indietro** quando una soluzione è impossibile
3. **Prova alternative** fino a trovare tutte le soluzioni

Nel nostro programma:

```
void InserisciQueen(int[] queen, List<int[]> solutions, int row) {  
    if (row == N) { // •CASO BASE: tutte le regine piazzate  
        // Salva la soluzione  
        return;  
    }  
  
    for (int col = 0; col < N; col++) {  
        if (IsSafe(queen, row, col)) {  
            queen[row] = col;                // Scendi  
            InserisciQueen(queen, solutions, row + 1); // Ricorsione  
            // Quando torna: torna indietro (backtracking)  
        }  
    }  
}
```

LINQ (Language Integrated Query)

Cos'è LINQ?

LINQ consente di scrivere query come SQL direttamente in C#. È **dichiarativo** (dici COSA fare, non COME).

Sintassi: Query Syntax vs Method Syntax

Query Syntax (stile SQL):

```
var risultato = from x in lista
  where x > 5
  select x * 2;
```

Method Syntax (stile funzionale):

```
var risultato = lista.Where(x => x > 5).Select(x => x * 2);
```

Metodi LINQ Usati nel Programma

1. `.Any()` - Verifica se esiste un elemento

```
bool Exists(int[] a)
=> solutions.Any(sol => sol.SequenceEqual(a));
```

Cosa fa: Ritorna `true` se ALMENO UN elemento soddisfa la condizione.

Esempio:

```
List<int> numeri = new List<int> { 1, 2, 3, 4, 5 };

bool hasPari = numeri.Any(x => x % 2 == 0); // true (2, 4 sono pari)
bool hasNegativo = numeri.Any(x => x < 0); // false
```

Nel nostro codice:

```
if (Exists(queen2) || Exists(VMirror(queen2))) {
  yesCopy = true; // Se esiste UNA simmetria
  break;
}
```

2. `.Select()` - Trasforma ogni elemento

```
int[] HMirror(int[] a)
=> a.Select(x => N - 1 - x).ToArray();
```

Cosa fa: Applica una funzione a ogni elemento e ritorna una nuova collezione.

Formula: `new_value = f(old_value)` per ogni elemento

Esempio:

```
int[] numeri = { 0, 1, 2, 3 };
int[] raddoppiati = numeri.Select(x => x * 2).ToArray();
// Risultato: { 0, 2, 4, 6 }

string[] nomi = { "Alice", "Bob" };
int[] lunghezze = nomi.Select(nome => nome.Length).ToArray();
// Risultato: { 5, 3 }
```

Nel nostro codice (HMirror):

```
int N = 8;
int[] colonne = { 0, 4, 7, 3, 2, 5, 1, 6 };
int[] specchiate = colonne.Select(x => N - 1 - x).ToArray();
// Converta: 07, 43, 70, 34, 25, 52, 16, 61
// Risultato: { 7, 3, 0, 4, 5, 2, 6, 1 }
```

3. `.Reverse()` - Inverte l'ordine

```
int[] VMirror(int[] a)
=> a.Reverse().ToArray();
```

Cosa fa: Inverte l'ordine degli elementi.

Esempio:

```
int[] numeri = { 1, 2, 3, 4 };
int[] invertiti = numeri.Reverse().ToArray();
// Risultato: { 4, 3, 2, 1 }

string[] parole = { "Ciao", "Mondo" };
parole.Reverse(); // { "Mondo", "Ciao" }
```

Nel nostro codice (VMirror):

```
int[] soluzione = { 0, 4, 7, 3, 2, 5, 1, 6 };  
int[] specchio = soluzione.Reverse().ToArray();  
// Risultato: { 6, 1, 5, 2, 3, 7, 4, 0 }
```

4. `.SequenceEqual()` - Confronta due sequenze

```
bool Exists(int[] a)  
=> solutions.Any(sol => sol.SequenceEqual(a));
```

Cosa fa: Ritorna `true` se due sequenze hanno gli stessi elementi nello stesso ordine.

Esempio:

```
int[] a = { 1, 2, 3 };  
int[] b = { 1, 2, 3 };  
int[] c = { 1, 2, 4 };  
  
a.SequenceEqual(b); // true (stesso contenuto)  
a.SequenceEqual(c); // false (diversi)  
a == b;             // false (riferimenti diversi)
```

Nel nostro codice:

```
solutions.Add(new int[] { 0, 4, 7, 3, 2, 5, 1, 6 });  
  
int[] check = { 0, 4, 7, 3, 2, 5, 1, 6 };  
if (solutions.Any(sol => sol.SequenceEqual(check))) {  
    Console.WriteLine("Soluzione già presente!");  
}
```

5. `.ToArray()` - Converta in array

```
int[] VMirror(int[] a)  
=> a.Reverse().ToArray();
```

Cosa fa: Converte qualsiasi `IEnumerable<T>` in array `T[]`.

Esempio:

```
List<int> lista = new List<int> { 1, 2, 3 };
int[] array = lista.ToArray();
// Risultato: int[] { 1, 2, 3 }

string testo = "ABC";
char[] caratteri = testo.ToCharArray(); // { 'A', 'B', 'C' }
```

Nel nostro codice:

```
var queen2 = queen.ToArray(); // Copia array
int[] speculare = VMirror(queen).ToArray();
solutions.Add(queen.ToArray()); // Aggiungi copia
```

Lambda Functions

Cos'è una Lambda?

Una **lambda** è una **funzione anonima** (senza nome) usata come argomento.

Sintassi: `(parametri) => corpo`

Esempi di Lambda nel Programma

1. Nel metodo `.Any()`:

```
solutions.Any(sol => sol.SequenceEqual(a))
// ^    ^ lambda
```

Leggi come: *"esiste una soluzione (sol) tale che è uguale ad a?"*

2. Nel metodo `.select()`:

```
a.Select(x => N - 1 - x)
// ^    ^ lambda: trasforma x in N - 1 - x
```

Leggi come: "per ogni elemento x , trasforma in $N - 1 - x$ "

Lambda con Multiple Righe

```
// Lambda a una riga
var raddoppio = numeri.Select(x => x * 2);

// Lambda a più righe (con block body)
var risultato = numeri.Select(x => {
    int doppio = x * 2;
    return doppio + 1;
});
```

Lambda vs Named Function

```
// Senza lambda: scrivi una funzione
bool IsPari(int x) {
    return x % 2 == 0;
}
var pari = numeri.Where(IsPari);

// Con lambda: inline
var pari = numeri.Where(x => x % 2 == 0);
```

Operazioni su Dati

`.Count()` - Conta gli elementi

```
int totale = solutions.Count;
```

Differenza:

- `.Count` (proprietà): accesso diretto, **più veloce** per liste
- `.Count()` (metodo LINQ): lavora con qualsiasi `IEnumerable<T>`

```
int[] arr = { 1, 2, 3 };
int c1 = arr.Count();           // LINQ (più lento)
int c2 = arr.Length;           // Proprietà (veloce)

List<int> lista = new List<int> { 1, 2, 3 };
```

```
int c3 = lista.Count;           // Proprietà (veloce)
int c4 = lista.Count();         // LINQ
```

.Where() - Filtra elementi

```
// Pseudocodice nel nostro programma:
var soluzioniPerOrdine = solutions
.Where(s => s[0] == 0) // Solo soluzioni che iniziano con regina in
colonna 0
.ToList();
```

Sintassi completa:

```
List<int> numeri = new List<int> { 1, 2, 3, 4, 5 };
var maggioriDi3 = numeri.Where(x => x > 3).ToList();
// Risultato: { 4, 5 }
```

.OrderBy() / .OrderByDescending() - Ordina

Nel programma (linea 16):

```
for (int i = 0; i < solutions.Count; i++) {
    Write($"Soluzione numero {numv,2} : ");
    // Potremmo ordinare:
    // solutions.OrderByDescending(s => s[0])
}
```

Esempio:

```
int[] soluzioni = { 0, 4, 7, 3, 2, 5, 1, 6 };

// Ordina per primo elemento (ascendente)
var ordinata = solutions.OrderBy(s => s[0]).ToList();

// Ordina per primo elemento (discendente)
var ordinata = solutions.OrderByDescending(s => s[0]).ToList();

// Ordina per somma di elementi
var ordinata = solutions.OrderBy(s => s.Sum()).ToList();
```

.Take() / .Skip() - Prende/Salta elementi

```
List<int> numeri = new List<int> { 1, 2, 3, 4, 5 };

// Prendi i primi 3
var primi3 = numeri.Take(3).ToList(); // { 1, 2, 3 }

// Salta i primi 2, prendi i prossimi 2
var nel_mezzo = numeri.Skip(2).Take(2).ToList(); // { 3, 4 }

// Ultimi 2
var ultimi2 = numeri.TakeLast(2).ToList(); // { 4, 5 }
```

.GroupBy() - Raggruppa elementi

```
// Raggruppa soluzioni per primo elemento
var gruppi = solutions.GroupBy(s => s[0]).ToList();

foreach (var gruppo in gruppi) {
    Console.WriteLine($"Colonna {gruppo.Key}: {gruppo.Count()}
soluzioni");
}
```

.Distinct() / .DistinctBy() - Elementi unici

Problema: Con array, `.Distinct()` confronta **riferimenti**, non **contenuto**.

```
int[] a = { 1, 2, 3 };
int[] b = { 1, 2, 3 };

new[] { a, b }.Distinct().Count(); // 2 (diversi oggetti)

// Soluzione: usa DistinctBy o Comparatore personalizzato
var unici = solutions
    .DistinctBy(s => string.Join(",", s)) // C# 9+
    .ToList();
```

.Any() VS .All() - Condizioni su collezioni


```
int[] numeri = { 2, 4, 6, 8 };

// Esiste almeno uno pari?
bool hasPari = numeri.Any(x => x % 2 == 0); // true

// Sono TUTTI pari?
bool allPari = numeri.All(x => x % 2 == 0); // true

int[] misti = { 1, 2, 3, 4 };
bool allPari2 = misti.All(x => x % 2 == 0); // false
```

Expression-Bodied Members

Nel programma usiamo la sintassi compatta con `=>`:

```
// Forma tradizionale
int[] VMirror(int[] a) {
    return a.Reverse().ToArray();
}

// Expression-bodied (più conciso)
int[] VMirror(int[] a)
=> a.Reverse().ToArray();
```

Quando usare:

- Logica semplice (una sola espressione)
- Evita con logica complessa (usa `{ }`)

Array e List: Differenze

Operazione	int[]	List<int>
Dimensione	Fissa	Dinamica
Accesso	<code>arr[0]</code>	<code>list[0]</code>
Count	<code>.Length</code>	<code>.Count</code>
Aggiungi	Non possibile	<code>.Add()</code>
Rimuovi	Non possibile	<code>.Remove()</code>
LINQ	Tutti i metodi	Tutti i metodi

Metodi Utili per Operare su Dati

Conversioni

```
int[] arr = { 1, 2, 3 };
List<int> lista = arr.ToList();           // Array ' List
int[] arr2 = lista.ToArray();             // List ' Array
string testo = string.Join(",", arr);    // Array ' String
```

Ricerche

```
int[] numeri = { 1, 2, 3, 4, 5 };

int primo = numeri.First();               // 1
int primo_pari = numeri.First(x => x % 2 == 0); // 2
int primo_o_default = numeri.FirstOrDefault(); // 1
int primo_o_default_pari = numeri.FirstOrDefault(x => x > 10); // 0
```

Aggregazioni

```
int[] numeri = { 1, 2, 3, 4, 5 };

int somma = numeri.Sum();                 // 15
double media = numeri.Average();          // 3
int max = numeri.Max();                   // 5
int min = numeri.Min();                   // 1
int prodotto = numeri.Aggregate((a,b) => a * b); // 120 (1*2*3*4*5)
```

Applicazioni nel Nostro Programma

1. IsSafe() - Verifica sicurezza della posizione

```
bool IsSafe(int[] queen, int row, int col) {
    for (int i = 0; i < row; i++) {
        // Controlla colonna e diagonali
        if (queen[i] == col || Math.Abs(queen[i] - col) == Math.Abs(i - row)) {
```

```

    return false;
}
}
return true;
}

```

2. Rotate() - Trasforma array geometricamente

```

int[] Rotate(int[] a) {
    int[] tmp = new int[N];
    for (int i = 0; i < N; i++)
        tmp[a[i]] = N - 1 - i; // Mappa nuova posizione
    return tmp;
}

```

3. Exists() - Controlla se soluzione è già presente

```

bool Exists(int[] a)
=> solutions.Any(sol => sol.SequenceEqual(a));

```

Riassunto LINQ nel Programma

Metodo	Uso	Risultato
<code>.Any()</code>	<code>solutions.Any(sol => ...)</code>	Verifica esistenza
<code>.Select()</code>	<code>a.Select(x => N - 1 - x)</code>	Trasforma elementi
<code>.Reverse()</code>	<code>a.Reverse()</code>	Inverte ordine
<code>.ToArray()</code>	<code>.ToArray()</code>	Converte a array
<code>.SequenceEqual()</code>	<code>sol.SequenceEqual(a)</code>	Confronta sequenze

Esercizi Proposti

1. Conta soluzioni per prima regina:

2. Trova soluzione con somma massima:

3. Soluzioni che iniziano con regina in colonna 0:

Fine Note